

* **INCAPSULARE** - mecanism prin care se restrictioneaza accesul la datele si metodele unei clase

- public: accesibile de oriunde din afara clasei, atat timp cat obiectul este vizibil
- private: accesibile doar din interiorul clasei la care sunt membre (modul implicit de acces in C++)
- protected: accesibile din interiorul clasei si din interiorul eventualelor clase derivate (mostenite)

* **THIS** - pointer ascuns care indica adresa obiectului curent din care s-a apelat o metoda

- este transmis automat ca argument suplimentar in toate functiile membre ale unei clase
- util pentru a rezolva conflicte de nume sau cand avem nevoie sa returnam insusi obiectul curent (ex: return *this)

* **TEMPLATE** - mecanism c++ pentru programare generica; permite scrierea unei singure functii sau clase pentru mai multe tipuri de date

- asigura refolosirea eficienta a codului

exemplu de definire (clasa):

```
template <class T>
class Stiva {
private:
    T* elemente;
    int dimensiune;
    int capacitate;
public:
    Stiva(int cap = 10) { ... }
};
```

* **OPERATOR OVERLOAD** - mecanism c++ care permite schimbarea semnificatiei unui operator standard (+, -, <<, =, etc.)

- ofera eleganta codului si permite folosirea obiectelor in expresii intuitive
- nu se pot inventa operatori noi, ci pot fi redefiniti doar cei existenti din C++ respectand aritatea (numarul de parametri)

exemplu de definire ca metoda membra (operatorul +=):

```
Stiva<T>& operator+=(const T& element) {
    if (dimensiune < capacitate) {
        elemente[dimensiune++] = element;
    }
    return *this;
}
```

* **FRIEND FUNCTIONS** - functii care sunt declarate externe clasei, dar care au dreptul sa acceseze datele private si protected ale acesteia

- se declara in interiorul clasei introducand cuvantul cheie "friend" inaintea definitiei functiei
- foarte utile mai ales cand vrem sa supraincarcam operatorii de introducere/extragere in flux (<<, >>)

* **VIRTUAL FUNCTIONS** - metode membre care permit implementarea corecta a polimorfismului si suprascrierea in clasele derivate

- asigura legarea dinamica la timpul executiei folosind tabela virtuala (Virtual Memory Table - VMT)
- cand o functie e virtuala, un apel printr-un pointer de tip clasa de baza va gasi si va rula in mod corect codul din clasa derivata in functie de ce adresa stocheaza pointerul in acel moment
- se declara adaugand cuvantul cheie "virtual" in fata antetului metodei din clasa de baza

* **PURE VIRTUAL FUNCTIONS** - functii virtuale care nu sunt implementate pe un anumit nivel (in clasa de baza)

- se declara adaugand pur si simplu sirul "= 0" la finalul antetului functiei virtuale
- obliga clasele derivate sa ofere ele codul propriu-zis pentru acea functie, altfel clasa derivata devine la randul ei abstracta

* **ABSTRACT CLASS** - o clasa care contine cel putin o functie pur virtuala

- nu poate fi instantiata (nu putem crea obiecte din ea) direct, generand o eroare la compilare; serveste doar drept "sablon"
- utila pentru a trasa specificul unei ierarhii de clase pe care dorim sa le modelam

exemplu de definire:

```
class Entitate {
public:
    virtual void afiseazaDetalii() const = 0;
};
```

* **INHERITANCE** - concept de baza POO prin care o clasa (derivata) preia sau mosteneste toate caracteristicile altei clase (de baza) pe care le poate particulariza

- limbajul C++ este unul dintre putinele limbaje care accepta mostenirea multipla (o clasa noua deriva simultan din mai multe clase de baza)
- clasele virtuale (derivarea virtuala) se folosesc pentru a preveni existenta de sub-obiecte duplicate daca o clasa mosteneste din doua clase care, la randul lor, provin din aceeasi radacina

exemplu de definire (multipla si virtuala):

```
class Produs : virtual public Entitate, public Printabil {
public:
    void afiseazaDetalii() const override { ... }
    void printeazaEticheta() const override { ... }
};
```

* **COPY CONSTRUCTOR** - o categorie speciala de constructor care initializeaza un obiect nou folosindu-se de o constanta referinta ce arata fix spre datele unui obiect deja existent

- scrierea lui este strict necesara cand clasa respectiva contine date alocate dinamic (pointeri cu new) pentru a garanta copierea corecta a instantelor
- daca nu scriem acest constructor manual, compilatorul va genera unul implicit care face "shallow copy", adica va copia doar adresa memoriei alocate; acest lucru cauzeaza probleme deoarece ambele instante isi vor imparti practic aceeasi adresa
- se apeleaza automat si cand returnam un obiect prin valoare dintr-o functie

exemplu de definire:

```
Produs::Produs(const Produs& altul) : pret(altul.pret) {  
    nume = new char[strlen(altul.nume) + 1];  
    strcpy(nume, altul.nume);  
}
```

* **STATIC MEMBERS** - attribute sau metode declarate statice care sunt folosite in comun de absolut toate obiectele (instantele) aceleiasi clase

- memoria pentru un membru static este alocata intr-o singura zona unica inca de la inceputul executiei intregului program
- modificarile aduse campului sunt vizibile in toate obiectele clasei si poate fi accesat direct chiar si in lipsa unui obiect, folosind direct operatorul de rezolutie (NumeClasa::MembruStatic)

exemplu de definire in interiorul clasei:

```
class Entitate {  
protected:  
    static int contorInstante;  
public:  
    static int getContor() { return contorInstante; }  
};
```

exemplu de definire, initializare (in afara descrierii clasei) si folosire:

```
int Entitate::contorInstante = 0;  
cout << Entitate::getContor();
```